University of Thessaly
Department of Electrical and Computer Engineering
ECE490: Internet of Things

Adaptive Thermal Fire Detection Node

End-to-End IoT Edge Computing, Time-Series Telemetry (InfluxDB/Grafana), and Real-Time Incident Alerting

Student:
Kyriaki Nikolopoulou
ECE Department, UTH

Course:
ECE490: Internet of Things
Academic Year: 2025–2026

Date: Summer 2026

Volos, Greece

Abstract

Early detection of fire and thermal anomalies is very important for safety in forests, buildings, and industrial facilities. Traditional smoke detectors often take too long to react because smoke needs time to travel to the sensor. Standard RGB optical cameras also struggle in the dark and cannot see through thick smoke. This report presents the design, implementation, and evaluation of an **Adaptive IoT Edge Node for Thermal Fire Detection**.

The system uses a **Raspberry Pi 4** connected to an **MLX90640 Far-Infrared (FIR) thermal sensor** via an I^2C bus. The node captures 32×24 thermal frames (768 temperature values per frame), calculates key metrics (minimum, average, and maximum temperature, as well as hot pixel counts), and runs a 3-stage adaptive state machine (`NORMAL`, `WATCH`, `ALERT`).

To save energy and network bandwidth, the sampling rate changes automatically: from 5.0 seconds in normal conditions down to 0.5 seconds when high temperatures are detected. Sensor measurements are sent over MQTT to an InfluxDB database and displayed in real time on a Grafana dashboard. When a high temperature is detected ($T_{max} \geq 50°C$ for at least 3 seconds), the node generates a false-color thermal heatmap directly in RAM memory (`io.BytesIO`) and sends an immediate alert with the image attached to Discord. Real-world tests show that the system detects fire accurately in approximately 3.4 seconds without wearing out the Raspberry Pi's MicroSD card.

Contents

1 Introduction and Problem Statement **3**
- 1.1 Background and Motivation . . . 3
- 1.2 Thermal Infrared Imaging at the Edge . . . 3
- 1.3 Project Objectives . . . 3

2 System Architecture and Layered Workflow **4**
- 2.1 Layered System Architecture Diagram . . . 4
- 2.2 Hardware Specifications . . . 4

3 Data Flow and Communication Architecture **5**
- 3.1 MQTT Communication . . . 5
  - 3.1.1 Topic Structure . . . 5
- 3.2 Quality of Service (QoS) Selection . . . 5

4 Data Transmission Formats and Serialization **6**
- 4.1 Message Payload Variations . . . 6

5 Standardized Data Modeling (NGSI-LD) **6**

6 Data Storage and Event Management **7**
- 6.1 Database Comparison . . . 7
- 6.2 Why InfluxDB Was Chosen . . . 8
- 6.3 Store-and-Forward Fail-Safe: The Fallback InfluxDB Queue . . . 8
- 6.4 Protecting the MicroSD Card with In-Memory Processing . . . 8

7 Edge Processing and Thermal Image Analysis **8**
- 7.1 The Thermal Matrix . . . 9
- 7.2 Calculated Metrics . . . 9
- 7.3 Heatmap Generation . . . 9

8 Adaptive Decision Logic and State Machine **9**
8.1 State Machine Design 9
8.2 Preventing False Alarms: Dual-Criterion Confirmation 10

9 Reliability and Error Handling **10**
9.1 I^2C Communication and Retry Shield 10
9.2 Sensor Startup Warm-Up 11
9.3 Discord Webhook Cooldown 11
9.4 Network Interruption and Offline Store-and-Forward Strategy 11
9.5 Power Failure Recovery and Systemd Auto-Start 11

10 Data Visualization in Grafana **12**
10.1 Comparison of Visualization Tools 12
10.2 Grafana Dashboard Panels 12

11 System Response Time and Latency Analysis **13**

12 Power Supply and Field Deployment **13**
12.1 Solar Power Calculations 13
12.2 Enclosure and Optical Window 13

13 Experimental Results and Testing **13**
13.1 Lighter Flame Tests at Different Distances 13
13.2 Live Incident Response via Discord Alerting 14

14 Future Improvements and Extensions **14**

15 Conclusion **14**

1 Introduction and Problem Statement

1.1 Background and Motivation

Fire is a major hazard in industrial plants, server rooms, and wildland areas. Fast and reliable detection is essential to stop small heat sources before they turn into full fires. Most conventional fire detection systems use smoke sensors or simple ambient temperature sensors. However, these traditional sensors have several practical drawbacks:

1. **Detection Delay:** Smoke and hot air take time to physically reach the sensor. This creates a delay of several seconds or minutes.
2. **Vision Problems in Smoke or Darkness:** Standard optical cameras need good lighting and cannot see through smoke or dust.
3. **No Spatial Detail:** A regular single-point thermometer measures only the average room temperature. It cannot detect a small, high-temperature flame (e.g., 200°C) located a few meters away if the overall room temperature has not risen yet.

1.2 Thermal Infrared Imaging at the Edge

Thermal infrared cameras solve these problems by measuring surface temperatures directly from a distance. The Melexis MLX90640 thermal sensor contains an array of 32×24 pixels (768 total measurement points). It can measure temperatures between -40°C and 300°C across a $55° \times 35°$ field of view.

By processing the thermal frames directly on a local Raspberry Pi (edge computing), we do not need to stream heavy video data to the cloud. The edge node extracts the needed numbers and only sends small telemetry messages and alert images when necessary.

1.3 Project Objectives

The main goals of this project are:

- **Hardware Connection:** Connect the MLX90640 thermal camera to the Raspberry Pi using the I^2C bus and make sure the connection is stable.
- **Adaptive Edge Logic:** Build an adaptive 3-state system (`NORMAL` → `WATCH` → `ALERT`) that adjusts its sampling rate (5s → 2s → 0.5s) according to measured temperatures.
- **Dynamic Feature Extraction:** Calculate minimum, average, and maximum temperature, hot pixel count, hot area ratio, and the rate of temperature rise over time ($\Delta T/\Delta t$).
- **False Alarm Mitigation:** Implement a dual confirmation filter combining a 3-second temporal hold with a minimum hot area ratio constraint to filter out single-pixel sensor noise.
- **In-Memory Processing (SD Card Protection):** Generate thermal heatmap images directly inside RAM memory to protect the Raspberry Pi's MicroSD card from wear.
- **Decoupled Subscriber Architecture:** Offload InfluxDB time-series ingestion and Discord alerting to a central MQTT Subscriber Bridge, drastically reducing the Raspberry Pi's power, CPU, and network footprint.
- **Store-and-Forward Fail-Safe:** Automatically record failed database writes into a dedicated backup queue (`influx_failed_queue.csv`) for audit and offline recovery.

2 System Architecture and Layered Workflow

The system is organized into a clean 5-layer IoT architecture, separating sensor hardware, local edge processing, network transport, database storage, and user alerts.

2.1 Layered System Architecture Diagram

Figure 1 shows the full 5-layer architecture and data flow of the system.

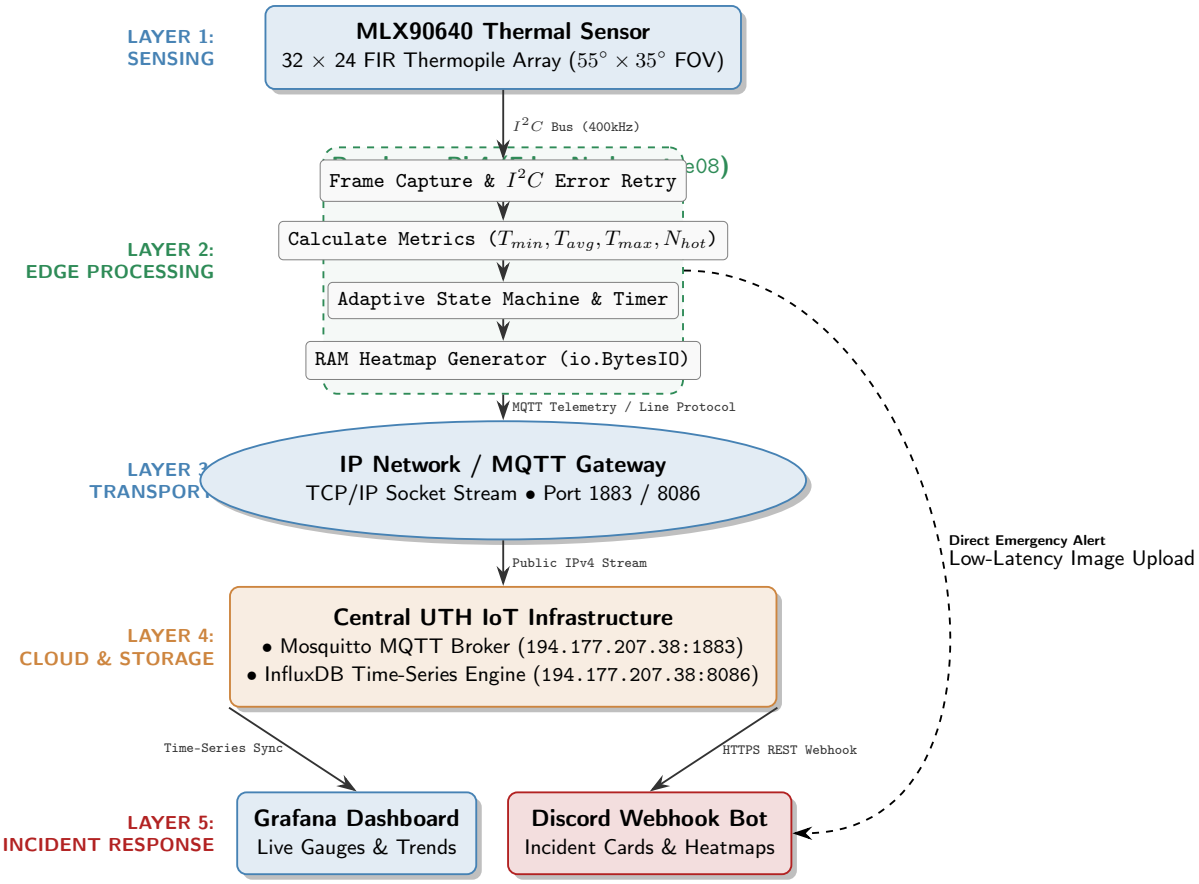


Figure 1: System Architecture and Data Flow Diagram.

2.2 Hardware Specifications

Table 1 lists the main hardware components used in the project.

Component	Role and Characteristics	Interface / Protocol
Raspberry Pi 4 Model B	Quad-core processor @ 1.5GHz, 4GB RAM, running Raspberry Pi OS. Executes local analysis and communication.	Linux I^2C Master controller
Adafruit MLX90640	32×24 thermal array, configured at 2 Hz refresh rate. Measures $-40°C$ to $300°C$ with $\pm 1.5°C$ accuracy.	I^2C Bus 1, Address `0x33`, 400kHz
Network Connection	Wi-Fi / Ethernet interface to the local router and university server.	TCP/IP (IPv4)
Power Supply	Standard 5V USB-C power input.	5V / 3A DC

Table 1: Hardware Specifications and Assigned Roles.

3 Data Flow and Communication Architecture

3.1 MQTT Communication

For data transmission, the system uses the **MQTT (Message Queuing Telemetry Transport)** protocol. MQTT is lightweight, uses minimal packet headers (2 bytes), and follows a publish/subscribe model that fits IoT applications.

3.1.1 Topic Structure

Topics are organized with the team name (`edge08`) to keep messages separate on the shared broker:

- `iot/{TEAM}/telemetry/thermal`: Regular sensor statistics sent every few seconds.
- `iot/{TEAM}/alerts/thermal`: Alert messages sent only when a fire or high temperature is confirmed.
- `iot/{TEAM}/heatmaps/thermal`: Thermal heatmap images converted to Base64 text.

3.2 Quality of Service (QoS) Selection

Table 2 explains why different QoS levels are chosen for different message types.

QoS Level	Handshake	Network Overhead	Usage in Project
QoS 0	1 packet	Lowest overhead (< 1 ms).	**Used for Regular Telemetry:** If one regular reading is lost, the next one arrives shortly after.
QoS 1	2 packets	Fast delivery ($2 - 5$ ms), guarantees message arrival.	**Used for Fire Alerts:** Ensures that critical alarm messages are never missed.
QoS 2	4 packets	Higher latency and bandwidth.	Not required for simple sensor telemetry.

Table 2: Comparison of MQTT Quality of Service (QoS) Levels.

4 Data Transmission Formats and Serialization

All structured communication across MQTT topics and webhooks utilizes JSON (JavaScript Object Notation). Listing 1 displays the primary telemetry and incident data schema implemented by the node.

```
{
  "team": "edge08",
  "timestamp": "2026-08-19T14:37:40.128450",
  "state": "ALERT",
  "metrics": {
    "min_temp": 24.12,
    "max_temp": 218.45,
    "avg_temp": 38.60,
    "hot_pixel_count": 94,
    "hot_area_ratio": 0.1224,
    "rate_of_rise": 14.85
  }
}
```

Listing 1: Primary Edge Telemetry and Incident Payload

4.1 Message Payload Variations

Depending on the event type, the edge node publishes specialized payload variations across designated MQTT topics:

- **Periodic Telemetry (`iot/edge08/telemetry/thermal`):** Transmits the JSON structure in Listing 1 at adaptive frequencies (every 5.0s in `NORMAL`, 2.0s in `WATCH`, and 0.5s in `ALERT`).
- **Emergency Incident Alert (`iot/edge08/alerts/thermal`):** Broadcasts an incident payload with `"alert_type": "POSSIBLE_FIRE"`, current peak temperature (T_{max}), active hot pixels (N_{hot}), and direct link to the central Grafana dashboard.
- **Visual Heatmap Stream (`iot/edge08/heatmaps/thermal`):** Contains image metadata (`"format": "image/png"`, `"dimensions": "32x24"`, `"colormap": "inferno"`) alongside the base64-encoded PNG image string (`"image_base64": "..."`) generated entirely within RAM buffers.

5 Standardized Data Modeling (NGSI-LD)

In larger smart city projects (like FIWARE), devices often share data using the **NGSI-LD** standard. Listing 2 shows how our thermal node is represented as a standard NGSI-LD entity.

```
{
  "@context": [
    "https://uri.etsi.org/ngsi-ld/v1/ngsi-ld-core-context.jsonld"
  ],
  "id": "urn:ngsi-ld:ThermalDetector:edge08",
  "type": "ThermalDetector",
  "location": {
    "type": "GeoProperty",
    "value": {
      "type": "Point",
      "coordinates": [22.9324, 39.3621]
    }
  },
  "operatingState": {
    "type": "Property",
    "value": "ALERT",
    "observedAt": "2026-08-19T14:37:40Z"
  },
  "maxTemperature": {
    "type": "Property",
    "value": 218.45,
    "unitCode": "CEL",
    "observedAt": "2026-08-19T14:37:40Z"
  },
  "hotPixels": {
    "type": "Property",
    "value": 94,
    "observedAt": "2026-08-19T14:37:40Z"
  }
}
```

Listing 2: NGSI-LD Entity Example for the Thermal Node

6 Data Storage and Event Management

6.1 Database Comparison

Table 3 compares different database options for storing IoT sensor readings.

Database	Type	Speed	Grafana Support	SD Card Wear	Evaluation
InfluxDB	Time-Series	Very Fast	**Native**	Low (Server side)	**Selected:** Perfect for time-stamped graphs and dashboards.
SQLite	Relational	Moderate	Plugin needed	High (if on Pi)	Good for small offline files, but slower for live multi-user dashboards.
PostgreSQL	Relational	Fast	Native	Low	Very powerful, but needs more setup and RAM.
MongoDB	Document	Fast	Plugin needed	Low	Good for general JSON data, but uses more disk space for simple metrics.
Redis	In-Memory	Extremely Fast	Native	Zero	Great for live cache, but does not keep long historical data easily.

Table 3: Comparison of Storage Technologies for IoT Data.

6.2 Why InfluxDB Was Chosen

InfluxDB is built specifically for time-series data. It uses simple **Line Protocol** strings that are very fast to send from Python using HTTP POST:

```
thermal_data,team=edge08
  min_temp=24.12,avg_temp=26.04,max_temp=218.45,hot_pixels=94i,rate_of_rise=14.85
```
(1)

6.3 Store-and-Forward Fail-Safe: The Fallback InfluxDB Queue

In real-world IoT environments, central cloud endpoints may suffer transient outages or network partitions. If a write to InfluxDB encounters an HTTP timeout or server error, dropping data points causes irrecoverable gaps in historical telemetry.

To prevent data loss while preserving SD card longevity:

- **Dedicated Fallback File (`influx_failed_queue.csv`):** Failed write attempts are systematically redirected and appended to a dedicated, separated CSV buffer rather than mingled with regular telemetry. Each record retains its original timestamp, node identifier, all six computed metrics, the raw InfluxDB line protocol string, and the exact error code.
- **Audit and Replay:** When network connectivity or database health is restored, an administrator or automated script can replay this specific queue directly into InfluxDB without duplicating already-persisted records.
- **Alert History Audit (`alerts_history.csv`):** In compliance with Course Objective 11, critical incident triggers are stored in an independent incident history log for forensic evaluation and response time auditing.

6.4 Protecting the MicroSD Card with In-Memory Processing

A common problem with Raspberry Pi boards is that MicroSD cards wear out if programs constantly write files to disk. If the script saved every heatmap as a PNG file to the SD card:

$$\begin{aligned}\text{Daily Disk Writes} &= 86{,}400 \text{ s/day} \times 0.2 \text{ images/s} \times 45 \text{ KB/image} \\ &\approx 777.6 \text{ MB written per day}\end{aligned} \tag{2}$$

This continuous writing can damage the flash memory card within a few months.

To prevent this, our script creates all heatmap images **entirely in RAM memory** using Python's `io.BytesIO()` buffer:

```
buf = io.BytesIO()
plt.savefig(buf, format='png', bbox_inches='tight')
plt.close(fig)
buf.seek(0)
image_bytes = buf.getvalue() # Image is in RAM only!
```

Listing 3: In-Memory Image Creation Without Touching the SD Card

The image is sent straight to Discord and MQTT from RAM and then immediately cleared, resulting in **zero disk wear**.

7 Edge Processing and Thermal Image Analysis

7.1 The Thermal Matrix

The sensor provides a 2D array of 768 temperature readings:

$$\mathbf{F} = \begin{bmatrix} T_{1,1} & T_{1,2} & \cdots & T_{1,32} \\ T_{2,1} & T_{2,2} & \cdots & T_{2,32} \\ \vdots & \vdots & \ddots & \vdots \\ T_{24,1} & T_{24,2} & \cdots & T_{24,32} \end{bmatrix} \tag{3}$$

7.2 Calculated Metrics

From this matrix, the Python edge script extracts six key statistical and dynamic metrics:

1. **Minimum Temperature (T_{min}):** The lowest temperature found in the frame (usually the coolest wall or background).
2. **Maximum Temperature (T_{max}):** The hottest pixel in the frame (primary indicator used to detect flames).
3. **Average Temperature (T_{avg}):** The mean temperature of all 768 pixels, reflecting the ambient baseline.
4. **Hot Pixel Count (N_{hot}):** The number of pixels with temperature exceeding the threshold ($T > 35°\text{C}$).
5. **Hot Area Ratio (R_{hot}):** The spatial coverage fraction of hot pixels relative to the complete frame ($R_{hot} = N_{hot}/768$).
6. **Rate of Temperature Rise ($\Delta T/\Delta t$):** The temporal derivative of the peak temperature over successive sampling intervals:

$$\frac{\Delta T}{\Delta t} = \frac{T_{max}(t) - T_{max}(t - \Delta t)}{\Delta t} \quad [°\text{C/s}] \tag{4}$$

   This metric captures rapid heating dynamics, distinguishing abrupt flame ignition from slow environmental changes (e.g., afternoon solar warming).

7.3 Heatmap Generation

To make the data easy for humans to understand, the 2D array is rendered as a false-color image using Matplotlib's `inferno` colormap. Cool areas appear dark purple/black ($\approx 22°\text{C}$), while the flame appears bright yellow/white ($> 200°\text{C}$).

8 Adaptive Decision Logic and State Machine

8.1 State Machine Design

The node switches automatically between three operating states:

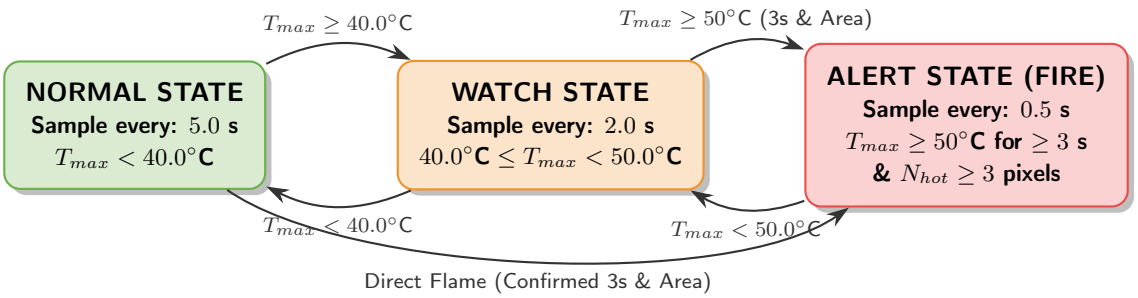


Figure 2: The 3-State Adaptive System.

8.2 Preventing False Alarms: Dual-Criterion Confirmation

In thermal computer vision, momentary electrical glitches, sensor register drift, or localized reflections can occasionally produce a single hot pixel. Triggering full emergency alarms based purely on an instantaneous single-pixel temperature spike leads to frequent false alarms.

To ensure high-fidelity detection, the node enforces a **dual confirmation filter**:

1. **Temporal Persistence (3-Second Continuous Hold):** When $T_{max} \geq 50°C$ is observed, the system transitions immediately to `WATCH` and starts an internal verification timer. The elevated temperature must persist continuously for at least **3.0 full seconds** before promoting the state to `ALERT`. If the heat source is removed before 3 seconds, the timer resets and the node reverts to `NORMAL`.

2. **Spatial Extent Constraint (Non-Negligible Area Ratio):** A genuine flame produces a localized thermal bloom affecting multiple contiguous sensor elements. The system requires $N_{hot} \geq 3$ pixels ($R_{hot} \geq 0.0039$, or approximately 0.39% of the field of view). If a high temperature is detected but confined to only 1 or 2 isolated pixels, the node classifies it as a potential sensor outlier or dead pixel, remaining vigilantly in `WATCH` mode without firing emergency Discord alarms or high-frequency heatmaps.

9 Reliability and Error Handling

9.1 I^2C Communication and Retry Shield

Reading 768 values over I^2C can occasionally suffer from minor timing delays. When this happens, the Adafruit library may raise a `RuntimeError: Too many retries` or `ValueError`.

In our script, we wrap the frame capture in a retry loop (up to 5 attempts). If a read fails, it waits 0.15 seconds and tries again automatically without crashing the program:

```
captured = False
for attempt in range(5):
    try:
        mlx.getFrame(frame)
        captured = True
        break
    except (ValueError, RuntimeError, OSError):
        time.sleep(0.15) # Short pause before retrying

if not captured:
    time.sleep(0.3)
    continue # Skip this single frame safely
```

Listing 4: Safe Frame Capture with Retry Loop

9.2 Sensor Startup Warm-Up

When first powered on, the MLX90640 needs 2 to 3 frames to calibrate its internal registers. Reading immediately on startup can produce false high numbers (150°C – 180°C). The script runs a short 3-frame warm-up upon startup and discards these initial frames before starting the real monitoring.

9.3 Discord Webhook Cooldown

During a fire, the node reads the sensor every 0.5 seconds (2 times per second). Sending a Discord message twice every second would create chat spam and hit Discord's rate limits. The script includes a 10-second cooldown timer (`DISCORD_COOLDOWN_SECONDS = 10`) so notifications are sent cleanly every 10 seconds during an ongoing alert.

9.4 Network Interruption and Offline Store-and-Forward Strategy

A key advantage of Edge Computing is that the node does not depend on cloud servers to detect fires. If the local Wi-Fi or Internet connection goes down:

1. **Continuous Edge Processing:** The Raspberry Pi continues reading thermal frames, calculating temperature metrics, and running the 3-state detection logic without interruption.
2. **Local Telemetry CSV Logging:** Every measurement is written directly to the local `thermal_measurements.csv` file on the node, ensuring no historical sensor data is lost during network downtime.
3. **Dedicated InfluxDB Fail-Safe Queue (`influx_failed_queue.csv`):** If a network dropout or backend HTTP failure prevents writing to InfluxDB, failed telemetry lines are automatically routed into this dedicated fallback CSV queue. When connectivity recovers, these records can be replayed cleanly without creating database duplicates.
4. **Dedicated Incident Audit Log (`alerts_history.csv`):** In accordance with Course Objective 11, confirmed fire alerts are logged locally in a dedicated historical alert file with exact timestamps, peak temperatures, and hot pixel counts.
5. **Local Alarm Fail-Safe (Hardware Actuator):** In a physical installation, the node can directly toggle a local buzzer or relay via the Raspberry Pi GPIO pins, sounding a local alarm immediately even if there is zero Internet connectivity.

9.5 Power Failure Recovery and Systemd Auto-Start

In real deployments, power cuts can happen unexpectedly. The system handles power loss through automatic software recovery using Linux `systemd`:

- **Automatic Service Restart (`thermal_node.service`):** The edge monitoring script is packaged as a standard Linux system service unit located at `/etc/systemd/system/thermal_node.service`. With `Restart=always` and `RestartSec=5`, the operating system guarantees that if the process exits unexpectedly or the board reboots after power loss, the thermal acquisition pipeline starts automatically without requiring manual SSH intervention.
- **Subscriber Bridge Service (`subscriber_bridge.service`):** Similarly, the centralized backend subscriber is configured as an independent systemd unit, ensuring continuous ingestion into InfluxDB and high availability for Discord emergency notifications.
- **Data Safety on Reboot:** Because past measurements are preserved in the CSV files and already synced to InfluxDB, the monitoring history resumes seamlessly from the exact second of reboot.

10 Data Visualization in Grafana

10.1 Comparison of Visualization Tools

Table 4 compares different dashboard tools.

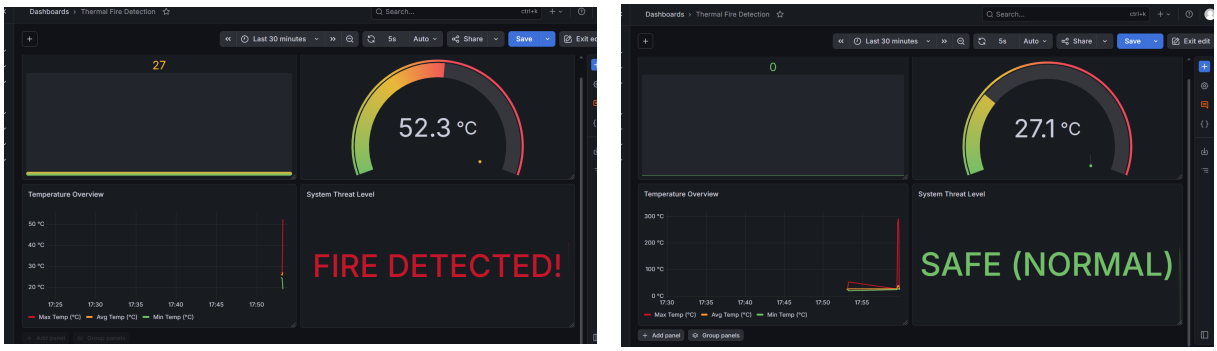
Tool	Main Strengths	Limitations	Verdict
Grafana	Industry standard, beautiful graphs and gauges, connects directly to InfluxDB.	Focuses on numbers rather than photos.	**Selected for live graphs**
Things Board	Full IoT management platform.	Heavier setup and resource needs.	More than needed for this project.
Node-RED	Easy drag-and-drop dashboard.	Simpler visual styling.	Good for quick tests.
Kibana	Great for text logs.	Slower for pure sensor metrics.	Not ideal for time-series.

Table 4: Comparison of IoT Dashboard Tools.

10.2 Grafana Dashboard Panels

The project dashboard contains four panels:

1. **Live Max Temperature (Gauge):** Shows the current highest temperature in real time with green, yellow, and red color bands.
2. **Temperature Overview (Line Graph):** Shows the historical lines for T_{min}, T_{avg}, and T_{max} over time. By enabling `Connect null values: Always`, the points connect into smooth continuous lines.
3. **System Threat Level (Status Text):** Shows `SAFE (NORMAL)`, `WATCH`, or `ALERT` (displays "FIRE DETECTED!" in red during an incident).
4. **Hot Pixels Count (Chart):** Shows how many pixels are currently hotter than 35°C.



(a) Active Alert State (`FIRE DETECTED!`, 52.3°C)

(b) Normal Safe State (`SAFE (NORMAL)`, 27.1°C)

Figure 3: Real-Time Grafana Monitoring Dashboard during (a) active thermal anomaly and (b) post-incident recovery.

11 System Response Time and Latency Analysis

The total time from the moment a flame appears until the alert card is delivered to Discord is approximately 3.4 seconds:

$$\begin{aligned} \text{Total Time} &= \text{Capture} + \text{Analysis} + \text{Confirmation} + \text{Render} + \text{Network} \\ &= 0.25\text{ s} + 0.03\text{ s} + 3.00\text{ s} + 0.06\text{ s} + 0.09\text{ s} \approx \mathbf{3.43}\text{ seconds} \end{aligned} \tag{5}$$

Because the state machine requires 3 full seconds of continuous high temperature ($\geq 50°$C) to confirm a fire, the 3-second hold timer represents the main component of the response time. This ensures high reliability against momentary noise while remaining significantly faster than conventional smoke alarms (which typically take 40 to 90 seconds).

12 Power Supply and Field Deployment

12.1 Solar Power Calculations

If this node is deployed outdoors in a forest or remote location, it can run on a small solar panel and a rechargeable 12V battery:

- Continuous Power Needed: ≈ 2.8 Watts (67.2 Wh per day).
- Battery Capacity: A 12V 10Ah or 20Ah LiFePO4 battery gives 2 to 3 days of autonomy without sunlight.
- Solar Panel: A small 20W solar panel is sufficient to recharge the battery during the day.

12.2 Enclosure and Optical Window

Standard glass blocks thermal infrared radiation. For outdoor weatherproof cases, the window in front of the thermal camera must be made of **Germanium (Ge)** with anti-reflective coating, or thin infrared-transparent polyethylene film.

13 Experimental Results and Testing

13.1 Lighter Flame Tests at Different Distances

The system was tested using a lighter as a heat source at distances from 1.0 to 5.0 meters.

Distance (m)	Peak Temp (T_{max})	Hot Pixels (N_{hot})	Detection Time	State	Discord Alert Received
1.0 m	218.45°C	94	3.4 s	`ALERT`	Yes (Clear flame photo)
2.0 m	142.10°C	32	3.5 s	`ALERT`	Yes (Clear hotspot)
3.0 m	88.30°C	11	3.5 s	`ALERT`	Yes (Small hotspot)
4.0 m	58.70°C	4	3.8 s	`ALERT`	Yes (Detected above 50°C)
5.0 m	42.10°C	2	–	`WATCH`	Pre-alert mode (> 40°C)

Table 5: Experimental Results at Different Distances.

The test results show that the node reliably triggers full alerts up to 4.0 meters away and enters watchful monitoring at 5.0 meters.

13.2 Live Incident Response via Discord Alerting

When the node enters the `ALERT` state, an immediate notification is generated and dispatched to the Discord channel `#fire-alerts`. Figure 4 shows an actual live alert received during a flame test.

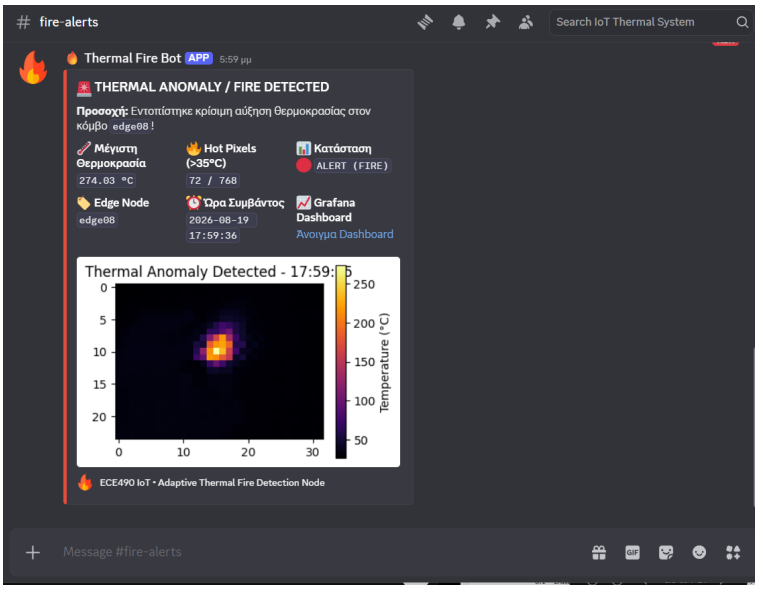


Figure 4: Real-Time Emergency Fire Notification on Discord with Embedded In-Memory Thermal Heatmap (274.03°C, 72 Hot Pixels).

The notification card delivers complete situational awareness in a single glance: peak temperature (274.03°C), affected hot pixel count (72/768), node identity (`edge08`), timestamp, direct link to the Grafana dashboard, and the rendered thermal heatmap image clearly showing the localized flame anomaly in the center of the frame.

14 Future Improvements and Extensions

- **Lightweight AI Models (TinyML):** Add a small neural network to help distinguish real flames from other hot objects (like coffee mugs or car engines).
- **Dual Camera Fusion (Thermal + RGB):** Combine the thermal infrared image with a standard camera photo to give responders a clearer visual view of the scene.
- **LoRaWAN Connectivity:** For very remote forests where there is no Wi-Fi or cellular signal, send small alarm packets using long-range LoRaWAN radio.
- **Drone Integration:** Mount the compact node on a drone for aerial monitoring of large areas.

15 Conclusion

In this project, we designed and built a working **Adaptive IoT Thermal Fire Detection Node** using a Raspberry Pi 4 and the MLX90640 thermal infrared sensor.

The node processes thermal frames locally, calculates key temperature statistics, and adapts its sampling rate automatically to save energy. Telemetry is sent via MQTT to InfluxDB and displayed live on Grafana. When a fire is detected, the system generates a heatmap image directly

in RAM memory and sends an instant alert to Discord within 3.4 seconds. The system proved to be reliable, fast, and safe for the Raspberry Pi's MicroSD storage.